

CS421 – Computer Networks

Programming Assignment 1

Fall 2025

Irmak İmdat

Files and the Logic

- **Day.py**

  This file includes a Day class for a day object to be used in the BookingManager.py file. As attributes, it includes the name of the day and a slot dictionary with slots as strings, paired with the user that booked the slot. “book_slot” function assigns the key of the selected slot with the user value after checking some constraints. These constraints include if the user made a reservation for that day already –if so, they can not make another reservation–, if the slot is valid, and if the slot is available. “cancel_slot” function checks the username values for a matching username; if it finds it, it switches the value of the pairing slot to “None”. Other functions are used for displaying purposes.

- **BookingManager.py**

  This file handles the main logic for the reservation of the court. It has a dictionary of usernames paired with user passwords, also a dictionary of day codes (string) paired with day objects. “make_booking” function first checks the validity of the user and the day, then uses the “book_slot” function of the day object to make the booking. “cancel_booking” makes the same constraint check, and then calls the “cancel_slot” function of the day object.

  It also has an “authanticate” function that checks whether a given username and password exist in the “users” dictionary attribute.

- **Server.py**

  This file implements the Server behavior. It creates a TCP socket (connection socket), binds it to the given address and port, and waits for connections. For each connection, it receives a single HTTP request, parses the request line and headers (parse_http_request), and routes the command (handle_command). Server’s logic in the “handle_request” method makes sure no command can be executed before the user logs in. When the user sends an HTTP request with “login”, “handle_login” runs, and it calls the “authenticate” function in the BookingManager.py file. If “authenticate” returns true, it creates a new token, adds it to the “SESSIONS”, and builds an HTTP response message with “Set-Auth-Token” header (later, requests are expected to include this token in the X-Auth-Token header.). One logic omitted here is log out. The user can login to another account without logging out from the current session. So long as the token is updated, there is no logical problem with implementing it like this.

Each command builds an HTTP response with a status line, a "Content-Type: text/plain" header, and a plain-text body. The server also validates inputs and returns appropriate error codes such as 400 Bad Request for malformed commands. URL-encoded command paths are decoded before the tokenization, so arguments like usernames, day codes, and time slots are correctly parsed.

- **Client.py**

This is the file where the client connects to the connection socket of the server. Since server is already active waiting for requests, client file makes the user choose a command first (along with parameters if necessary). Then, based on this selection, it builds an HTTP message (except for "quit", quit breaks the "while True" loop), connects to the host and port (previously specified as a command line), and sends the request of the client after encoding it. It then receives the response and prints it. If the response coming from the server includes a token, it saves that as well for later commands to work (it includes the token as a header in the HTTP message).

Error Handling

Error handling is used when needed, for example the "handle_request" function in Server.py file (which calls all other functions in the file) includes an exception handling mechanism if something goes wrong building or parsing HTTP message, etc.

Decisions

I divided the actual reservation functionality to BookingManager and Day files, and I tried to keep the Server and Client files as simple as possible, in terms of simplicity, readability, and reusability. To do this, I first designed the logic of BookingManager and Day files, then all I needed to do was to focus on HTTP and TCP logic in Server and Client files. As I have mentioned earlier I omitted the log out logic and a user can use log in function when they are already logged in.